

SMART SPLIT-FEDERATED LEARNING OVER NOISY CHANNELS FOR EMBRYO IMAGE SEGMENTATION

Zahra Hafezi Kafshgari, Ivan V. Bajić, and Parvaneh Saeedi

School of Engineering Science, Simon Fraser University, Burnaby, BC, Canada

ABSTRACT

Split-Federated (SplitFed) learning is an extension of federated learning that places minimal requirements on the clients' computing infrastructure, since only a small portion of the overall model is deployed on the clients' hardware. In SplitFed learning, feature values, gradient updates, and model updates are transferred across communication channels. In this paper, we study the effects of noise in the communication channels on the learning process and the quality of the final model. We propose a smart averaging strategy for SplitFed learning with the goal of improving resilience against channel noise. Experiments on a segmentation model for embryo images shows that the proposed smart averaging strategy is able to tolerate two orders of magnitude stronger noise in the communication channels compared to conventional averaging, while still maintaining the accuracy of the final model.

Index Terms— Federated learning, split learning, communication noise, embryo image segmentation

1. INTRODUCTION

Federated learning (FL) [1] enables training of machine learning models without sharing data. This is important in certain scenarios, including medical data analysis, where data cannot be shared due to privacy or other regulations. However, FL may still be challenging for healthcare applications, because it assumes that participating clients – medical clinics in this case – have the computing infrastructure to train large machine learning models. During FL, each client is responsible for training a local model, which is then sent to a central server where local models are aggregated into a global model [2, 3]. The reality is that many medical clinics do not have the computing infrastructure for training local models, which are of the same size as the global model.

On the other hand, split learning (SL) [4] is a learning approach capable of addressing computational power imbalance between clients' devices and servers. In SL, the front-end of the model, which is usually a small part of the overall model, is located at the client's device, while the larger back-end is located at the server. During training, features (also known

as “smashed data”) are sent from the front-end to the back-end, while gradient updates are sent from the back-end to the front-end. SL can be used in a federated scenario across multiple clients [5, 6], in which case it is sometimes referred to as split-federated (SplitFed) learning [7]. SplitFed learning allows both privacy control and computational balancing between the clients and the server [8, 9]. However, it requires more communication than FL, because not only local models, but also features and gradient updates in each training batch need to be exchanged between the clients and the server.

No communication channel is perfect. Therefore, the data exchanged between the clients and the server will be noisy, and this will impact the SplitFed learning process. Specifically, noise in the features and gradient updates during local model training will degrade the quality of local models, whereas noise in the model transfer will impact the global model. To our knowledge, these issues were not previously studied in the context of SplitFed learning. In this paper, we examine the impact of communication noise on SplitFed learning of an embryo image segmentation model, and propose an averaging strategy – termed *Smart SplitFed* – that minimizes its effects. The paper is organized as follows. In Section 2, we present a brief overview of related work. The proposed method is described in Section 3. Experiments are presented in Section 4 followed by conclusions in Section 5.

2. RELATED WORKS

The key step in FL is aggregation (averaging) of local models into a global model. The simplest way of doing this is to take arithmetic average of local models' weights to update global model weights [1]. However, this simple strategy is agnostic to client heterogeneity, for example, different distribution of data across clients. Hence, several approaches were developed to address this issue.

In [10], an approach called Fed-ensemble was introduced to average K randomly trained models in the federation step to improve the generalization of the global model. In [11], the authors study robust heterogeneous FL including clients with mislabeled data. Note that in [11], data labels are considered to be subject to noise, whereas communication links are assumed to be perfect. Hence, [11] is complementary to the scenario studied here. Along similar lines, [12] created corrupted clients by randomly shuffling labels and adding

This work was supported in part by the NSERC grants RGPIN-2018-05164, RGPIN-2021-02485, and RGPAS-2021-00038.

Gaussian noise to input data, then minimized a weighted sum of empirical risks to alleviate the effects of corruption. It was shown that global performance drops drastically as more clients become corrupted. Yang *et al.* [13] also considered noisy labels in FL and proposed sharing centroids of local feature vectors for model aggregation and label correction. In [3], an averaging method called FedProx was proposed to deal with device and data heterogeneity in FL. None of these approaches considered communication noise.

FL was combined with split learning to create SplitFed in [7], to address both privacy issues and computational imbalance between the clients and the server. Two versions of SplitFed were proposed: vanilla SplitFed, in which the labels are shared with the server while the input data remains at the client; and U-shaped SplitFed (Fig. 1), in which both input data and labels remain at the client. Simple averaging was used for both versions of SplitFed in [7], which, as our results will show, is very sensitive to communication noise.

SplitFed is gaining popularity in healthcare applications. Poirot *et al.* [9] utilized it in experiments on fundus photos and chest X-rays. Gawali *et al.* [8] reviewed FL, SL, and SplitFed approaches in healthcare, and introduced another version of SplitFed called SpltFedv3, in which labels are shared, but client models are not averaged. Instead, only server-side model weights are averaged. In [5], SplitAVG was proposed for the purpose of averaging features sent to the server from heterogeneous clients. Both vanilla and U-shaped SplitFed were examined in [5]. In [6], heterogeneity in terms of energy harvesting capabilities of clients was tackled. However, none of these earlier works considered communication noise in SplitFed learning, which is our main focus.

3. PROPOSED METHOD

3.1. Segmentation model

We study training of a SplitFed U-Net, shown in Fig. 1, for segmenting embryo images. The overall U-Net consists of five downsampling blocks between the input and the bottleneck, and five upsampling blocks between the bottleneck and the output. Each block contains two convolutional layers with the kernel size of 3×3 , followed by batch normalization and ReLU activation. Each downsampling block ends with a 2×2 max-pooling layer; the number of filters in downsampling blocks is 32, 64, 128, 256, and 512, respectively. Each upsampling block starts with a 2×2 upsampling layer; the number of filters in upsampling blocks is 512, 256, 128, 64, and 32, respectively. The last convolutional layer is followed by an output `argmax` layer. The model is split at the first and last convolutional layer, as shown in the figure, so that the first convolutional layer (front-end, FE) and the last convolutional layer together with the output `argmax` layer (back-end, BE) are on the client side, while the rest of the model is on the server. Links from the client to the server (uplinks) are shown in black, while the links from the server to the client (down-

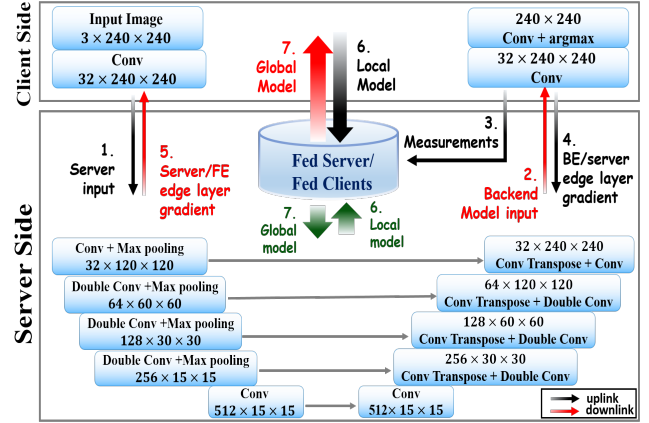


Fig. 1. SplitFed U-Net used in our work.

links) are shown in red.

3.2. Training procedure

The system adopts sequential training where the server trains with each of the N clients in turn. The server runs several *local epochs* of training with a specific client. At the end of this process, both the client and the server store their weights from the best local epoch, i.e., the local epoch with the lowest validation loss. Let W_i^S be the server weights from the best local epoch while training with client i , and $W_i^C = \{W_i^{FE}, W_i^{BE}\}$ be the weights of client i , which include both FE and BE weights, from the same epoch. One pass across all clients is called a *global epoch*. At the end of each global epoch, W_i^S and W_i^C are averaged into $\bar{W}^S$ and $\bar{W}^C$, respectively, to create a global model $W = \{\bar{W}^S, \bar{W}^C\}$ before the next global epoch starts. Averaging is discussed in Section 3.3 below.

Within each local epoch, in the forward pass, features produced by the FE are sent to the server, then processed by the server-side part of the model, then features from the server are sent to the client-side BE. In the backpropagation pass, gradient updates from the BE are sent to the server, then backpropagated through the server-side model, then sent to the FE. Data that is exchanged between the server and the client – features in the forward pass and gradients in backpropagation – are subject to noise during transmission.

3.3. Averaging

Two popular model averaging methods in federated learning are naive averaging and federated averaging [1]. In naive averaging, each client receives the same weight, so with N clients the averaging would look like:

$$\bar{W}^S = \frac{1}{N} \sum_{i=1}^N W_i^S, \quad \bar{W}^C = \frac{1}{N} \sum_{i=1}^N W_i^C. \quad (1)$$

In federated averaging, the weights are proportional to the number of data samples at each client. If m_i is the number of data samples at client i , federated averaging would be:

$$\bar{W}^S = \frac{1}{\sum_i m_i} \sum_{i=1}^N m_i W_i^S, \quad \bar{W}^C = \frac{1}{\sum_i m_i} \sum_{i=1}^N m_i W_i^C. \quad (2)$$

Neither of these popular averaging strategies takes into account the quality of clients' models. However, clients with noisy links tend to train less accurately than those with clean links, as our results will show. Hence, a "smart" averaging strategy should take that into account. Here we propose one such strategy whose goal is to take both potential data imbalance (different m_i 's) and client's loss (which is impacted by communication noise) in the averaging process.

After completing local epochs with client i and selecting the model from the best epoch, it is possible to compute the loss on each training sample j on that client, $\mathcal{L}_i^j$. Hence, the following statistics can be computed:

$$\mu_i = \text{mean}\{\mathcal{L}_i^1, \dots, \mathcal{L}_i^{m_i}\}, \quad \sigma_i = \text{std}\{\mathcal{L}_i^1, \dots, \mathcal{L}_i^{m_i}\}. \quad (3)$$

We want to assign lower weight to the clients with higher loss. However, the average loss estimate μ_i based on a relatively small m_i is unreliable. Hence, to reflect this uncertainty, an upper bound of a confidence interval for the mean loss can be taken as an indicator for the (un)reliability of that client. If $\mathcal{L}_i^j$'s were normally distributed, the upper bound of the 95% confidence interval for the mean loss would be

$$b_i = \mu_i + 2\sigma_i. \quad (4)$$

This is what we take as the indicator of (un)reliability of client i . All clients' indicators are collected in the vector $\mathbf{b} = [b_1, \dots, b_N]^\top$, then the vector of quality scores $\mathbf{q} = [q_1, \dots, q_N]^\top$ is computed as

$$\mathbf{q} = \text{softmax}[\alpha \cdot (1 - \mathbf{b})], \quad (5)$$

where α is a hyperparameter, set to $\alpha = 10$ in our experiments. The key point about (5) is that (un)reliability indicators in $\mathbf{b}$ appear with the negative sign, so that less reliable clients (higher b_i) receive lower quality scores (lower q_i). softmax serves to emphasize the more reliable clients.

Besides quality scores, we also take the data distribution across clients into account, in the same way as federated averaging does. Specifically, let $\mathbf{d} = [m_1, \dots, m_N]^\top / (\sum_i m_i)$ be the vector containing relative amounts of data at each client. Our final weights $\mathbf{r} = [r_1, \dots, r_N]^\top$ are obtained by combining quality scores and data distribution as follows:

$$\mathbf{r} = \frac{\mathbf{q} \odot \mathbf{d}}{\mathbf{q}^\top \mathbf{d}}, \quad (6)$$

where $\odot$ is the Hadamard product. Note that $\sum_i r_i = 1$. Finally, *Smart SplitFed* averaging is performed as:

$$\bar{W}^S = \sum_{i=1}^N r_i W_i^S, \quad \bar{W}^C = \sum_{i=1}^N r_i W_i^C. \quad (7)$$

4. EXPERIMENTS

4.1. Experimental setup

We train the SplitFed U-Net model (Fig. 1) on multi-label segmentation of human blastocyst components [14, 15]. There are 815 embryo images in total, each with a segmentation mask that includes five components: Background (BG), Zona Pellucida (ZP), Trophoctoderm (TE), Blastocoel (BL), and Inner Cell Mass (ICM). A test set of 100 images is saved for testing the global model. There are $N = 5$ clients and the data is non-uniformly distributed across clients, with 210, 120, 85, 180 and 120 samples respectively on clients 1, 2, ..., 5. Each client uses 85% of its data for training and 15% for validation. Clients gets trained over 12 local and 10 global epochs.

Input images are resized to 240×240 , and augmentation is performed using horizontal and vertical flipping, and rotation of up to $\pm 35^\circ$. Dice coefficient [16] is chosen as the loss function, and the system is trained with the Adam optimizer with the initial learning rate set to 10^{-3} . Inspired by Softcast [17], channel noise is simulated by adding white Gaussian noise with zero mean and standard deviation of σ_{noise} to all transmitted and received data of clients 3, 4, and 5. This means that noise is added to features transmitted in the forward pass, gradient updates in the backwards pass, as well as b_i and W_i^C (sent from client i to the server after local epochs) and $\bar{W}^C$ (sent from server to all clients after averaging, but contaminated only for noisy clients). Noise is added starting at global epoch 5 for client 3, global epoch 4 for client 4, and global epoch 3 for client 5.

4.2. Results

We compare the Naive SplitFed (1), SplitFedAVG (2) and Smart SplitFed (7) in terms of the global model accuracy and loss value at the end of the 10th global epoch. The results are shown in Table 1 in terms of the Intersection Over Union (IOU) of the five embryo segments and the overall accuracy, defined as the percentage of correctly classified pixels. The best results are shown in bold. Experiments were conducted for seven values of σ_{noise} shown in the table. In cases where the loss is shown as 'nan', the training did not converge, likely due to gradient noise contamination during back-propagation. This happens to Naive SplitFed and SplitFedAVG with stronger noise. In such cases, the model defaults to predicting only the background, and the IOU of other components is zero.

With $\sigma_{\text{noise}} = 0$, all averaging methods perform comparably, and similar to BLAST-Net [14]. However, as the noise increases, the advantage of Smart SplitFed over the other two methods becomes obvious. Naive SplitFed fails to converge already at $\sigma_{\text{noise}} = 10^{-3}$, while SplitFedAVG fails to converge starting with $\sigma_{\text{noise}} = 10^{-2}$. Smart SplitFed, on the other hand, is very robust to noise and barely loses any accuracy all the way up to $\sigma_{\text{noise}} = 5 \cdot 10^{-1}$.

To probe further, Fig. 2 shows the evolution of training at $\sigma_{\text{noise}} = 6 \cdot 10^{-4}$, because this is the highest noise level in the

Table 1. A comparison of Naive SplitFed, SplitFedAVG (AVG), and Smart SplitFed on the common test set at different values of σ_{noise} . Acc is the overall accuracy; the last five columns show IOU of Background (BG), Zona Pellucida (ZP), Trophoctoderm (TE), Inner Cell Mass (ICM), and Blastocoel (BL). Loss being ‘nan’ indicates that the method failed to converge.

σ_{noise}	Method	Loss	Acc	BG	ZP	TE	ICM	BL
0	Naive	0.08	93.39	0.93	0.78	0.74	0.85	0.88
	AVG	0.10	92.50	0.92	0.75	0.74	0.83	0.87
	Smart	0.06	93.60	0.94	0.80	0.75	0.85	0.88
$2 \cdot 10^{-4}$	Naive	0.17	89.00	0.90	0.71	0.70	0.73	0.79
	AVG	0.13	89.44	0.88	0.71	0.71	0.77	0.83
	Smart	0.07	93.52	0.95	0.78	0.73	0.83	0.88
$6 \cdot 10^{-4}$	Naive	0.21	82.02	0.74	0.51	0.65	0.75	0.79
	AVG	0.16	87.10	0.86	0.64	0.65	0.74	0.79
	Smart	0.07	93.18	0.94	0.77	0.72	0.83	0.87
10^{-3}	Naive	nan	40.27	0.40	0.00	0.00	0.00	0.00
	AVG	0.23	76.49	0.59	0.50	0.67	0.73	0.78
	Smart	0.12	92.97	0.93	0.78	0.74	0.83	0.87
10^{-2}	Naive	nan	40.27	0.40	0.00	0.00	0.00	0.00
	AVG	nan	40.27	0.40	0.00	0.00	0.00	0.00
	Smart	0.08	93.05	0.94	0.79	0.74	0.85	0.88
10^{-1}	Naive	nan	40.27	0.40	0.00	0.00	0.00	0.00
	AVG	nan	40.27	0.40	0.00	0.00	0.00	0.00
	Smart	0.08	92.66	0.93	0.78	0.74	0.85	0.86
$5 \cdot 10^{-1}$	Naive	nan	40.27	0.40	0.00	0.00	0.00	0.00
	AVG	nan	40.27	0.40	0.00	0.00	0.00	0.00
	Smart	0.06	93.12	0.94	0.78	0.73	0.82	0.87

experiments where all averaging strategies converged. Noise is introduced at global epochs 3, 4, and 5 (i.e., after local epochs 36, 48, and 60) at clients 5, 4, and 3. We can observe the corresponding increase in the local training loss at those clients in the top-row graphs in Fig. 2. The middle-row graphs show the clients’ weights used in averaging. Naive SplitFed uses equal weights, whereas SplitFedAVG uses weights proportional to the amount of data at each client. Neither of the two methods adjusts its weights during training. However, Smart SplitFed adjusts to the increased loss of clients 3, 4, and 5 by reducing their weights. Indeed, after global epoch 5, only clients 1 and 2 (with clean communication links) receive weights that are significantly different from zero. The result is the lower test loss at the end of training, as shown in the bottom-row graphs, as well as Table 1.

Finally, in Fig. 3 we visualize some of the segmentation masks. The top row shows an input image and the corresponding ground-truth segmentation mask with the five segmentation components indicated on the mask. The bottom row shows the predicted segmentation masks using models trained at $\sigma_{\text{noise}} = 6 \cdot 10^{-4}$ by Naive SplitFed, SplitFedAVG, and Smart SplitFed. We note that the model trained using Smart SplitFed has produced the most accurate prediction. In particular, its prediction about the size and shape of the ICM is clearly better than those produced by the models trained with Naive SplitFed and SplitFedAVG.

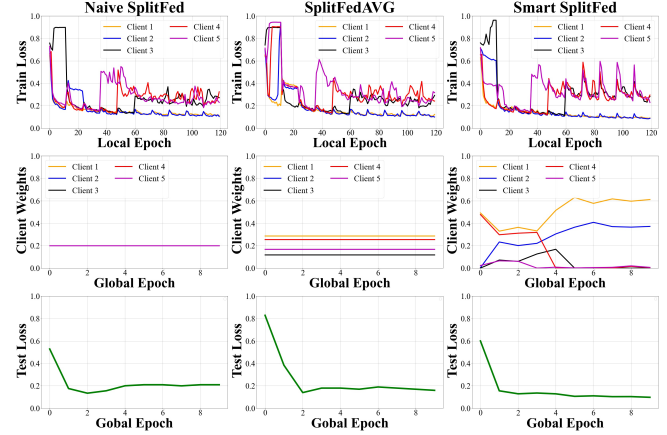


Fig. 2. A comparison of Naive SplitFed, SplitFedAVG, and Smart SplitFed at $\sigma_{\text{noise}} = 6 \cdot 10^{-4}$. Top row: training loss at each client. Middle row: clients’ weights used in averaging. Bottom row: test loss at each global epoch.

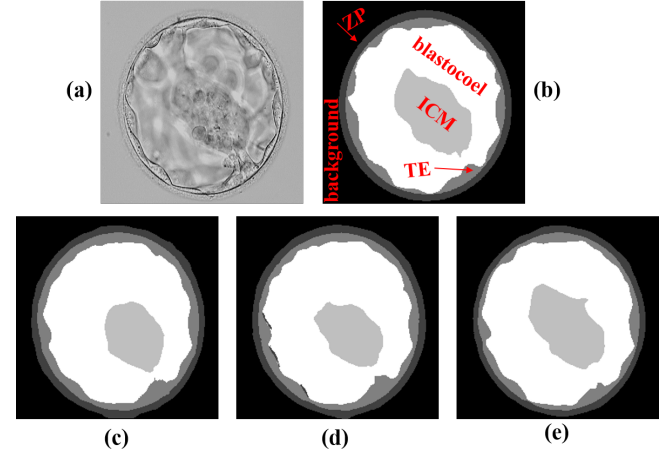


Fig. 3. (a) Input image, (b) ground truth segmentation, and predictions made by models trained at $\sigma_{\text{noise}} = 6 \cdot 10^{-4}$ using (c) Naive SplitFed, (d) SplitFedAVG, and (e) Smart SplitFed.

5. CONCLUSIONS

The focus of this paper was split-federated (SplitFed) learning in the presence of noise on the links between clients and the server. Traditional averaging methods were not designed for such a scenario, and our experiments on embryo image segmentation showed they are fairly fragile in this context. We proposed a novel averaging method called *Smart SplitFed*, which suppresses the effects of communication noise by monitoring the local loss values at each client and adjusting the averaging weights accordingly. The proposed method was shown to be much more robust to noise on the communication links compared to conventional methods and lead to more accurate global models.

6. REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019.
- [3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. Machine Learning and Systems*, vol. 2, pp. 429–450, 2020.
- [4] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.
- [5] M. Zhang, L. Qu, P. Singh, J. Kalpathy-Cramer, and D. L. Rubin, "Splitavg: A heterogeneity-aware federated deep learning method for medical imaging," *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 9, pp. 4635–4644, 2022.
- [6] X. Chen, J. Li, and C. Chakrabarti, "Energy and loss-aware selective updating for splitfed learning with energy harvesting-powered devices," *Journal of Signal Processing Systems*, pp. 1–15, 2022.
- [7] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "Splitfed: When federated learning meets split learning," in *Proc. AAAI Conference on Artificial Intelligence*, 2022, pp. 8485–8493.
- [8] M. Gawali, C. S. Arvind, S. Suryavanshi, H. Madaan, A. Gaikwad, K. N. Bhanu Prakash, V. Kulkarni, and A. Pant, "Comparison of privacy-preserving distributed deep learning methods in healthcare," in *Proc. Annual Conf. Medical Image Understanding and Analysis*, 2021, pp. 457–471.
- [9] M. G. Poirot, P. Vepakomma, K. Chang, J. Kalpathy-Cramer, R. Gupta, and R. Raskar, "Split learning for collaborative deep learning in healthcare," *arXiv preprint arXiv:1912.12115*, 2019.
- [10] N. Shi, F. Lai, R. A. Kontar, and M. Chowdhury, "Fed-ensemble: Improving generalization through model ensembling in federated learning," *arXiv preprint arXiv:2107.10663*, 2021.
- [11] X. Fang and M. Ye, "Robust federated learning with noisy and heterogeneous clients," in *Proc. IEEE/CVF CVPR*, 2022, pp. 10072–10081.
- [12] S. Li, E. Ngai, F. Ye, and T. Voigt, "Auto-weighted robust federated learning with corrupted data sources," *ACM Trans. Intelligent Systems and Technology*, vol. 13, no. 5, pp. 1–20, Oct. 2022.
- [13] S. Yang, H. Park, J. Byun, and C. Kim, "Robust federated learning with noisy labels," *IEEE Intelligent Systems*, vol. 37, no. 2, pp. 35–43, 2022.
- [14] R. M. Rad, P. Saeedi, J. Au, and J. Havelock, "BLAST-NET: Semantic segmentation of human blastocyst components via cascaded atrous pyramid and dense progressive upsampling," in *Proc. IEEE ICIP*, 2019, pp. 1865–1869.
- [15] P. Saeedi, D. Yee, J. Au, and J. Havelock, "Automatic identification of human blastocyst components via texture," *IEEE Trans. Biomedical Engineering*, vol. 64, no. 12, pp. 2968–2978, 2017.
- [16] R. Zhao, B. Qian, X. Zhang, Y. Li, R. Wei, Y. Liu, and Y. Pan, "Rethinking dice loss for medical image segmentation," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2020, pp. 851–860.
- [17] S. Jakubczak and D. Katabi, "SoftCast: One-size-fits-all wireless video," in *Proc. ACM SIGCOMM*, 2010, pp. 449–450.